

Assignment 3&4

Report

Νικολέτα Ξενάκη , csd4968

Γενικά

Και στις δύο υλοποιήσεις εφαρμόστηκαν οι βασικές έννοιες του Αντικειμενοστραφή Προγραμματισμού και για αυτό τον λόγο υπάρχουν structs (c++)/ κλάσεις (java) που αναπαριστούν: τον κάθε πλανήτη (Celestial body) , τις περιοχές (quads) του 2D επιπέδου και τους κόμβους του τετραδικού δένδρου(Node). Σε κάθε struct /class υπάρχουν και οι κατάλληλες συναρτήσεις για την διαχείριση των πεδίων και την επίτευξη ορισμένων λειτουργιών (όπως πχ η insert() στο δένδρο). Και στις δύο υλοποιήσεις η κατασκευή του δένδρου έγινε σειριακά, διότι στην παράλληλη εκδοχή θα χρειαζόμασταν locks κάτι το οποίο θα μείωνε δραματικά την απόδοση λόγω του μεγάλου contention για την παράλληλη επεξεργασία του δένδρου από τα νήματα. Έπειτα ο υπολογισμός του κέντρου μάζας για το κάθε κόμβο του δένδρου γίνεται παράλληλα ή σειριακά, καθώς ξεκινώντας από την ρίζα του δένδρου κάθε κόμβος καλεί αναδρομικά την συνάρτηση για κάθε ένα από τα 4 υπο-δένδρα του και στο τέλος αφού έχουν επιστρέψει όλες οι αναδρομές συνδυάζει τα αποτελέσματα των κλήσεων και αποθηκεύει το τελικό αποτέλεσμα στα πεδία του. Τέλος η φάση για τον υπολογισμό των δυνάμεων και την νέα θέση κάθε αντικειμένου γίνεται επίσης παράλληλα μεταξύ των πλανητών. Τα παραπάνω επαναλαμβάνονται για κάθε επανάληψη της προσομοίωσης με κάθε φορά να κατασκευάζεται και να χρησιμοποιείται νέο δένδρο.

Υλοποίηση με TBB

Εν συνεχεία των παραπάνω ο παραλληλισμός με TBB επιτεύχθηκε χρησιμοποιώντας:

tbb::parallel_invoke(); στα 4 υποδένδρα κάθε κόμβου, για τον παράλληλο υπολογισμό του κέντρου μάζας του.

tbb::parallel_for(); για τον υπολογισμό των δυνάμεων και την νέα θέση κάθε πλανήτη, διαμοιράζοντας τους στα νήματα.

Μετρήσεις για TBB:

Για κάθε input ισχύει:

5 runs.

Αριθμός επαναλήψεων προσομοίωσης = 10.000

Με χρήση της εντολής time παίρνοντας το real time.

Input1.txt

Threads

	1	2	4
Average Execution Time	0.26 sec.	0.33 sec.	0.41 sec.
Variance	0.006 sec.	0.013 sec.	0.003 sec.
Average Speedup	0	0.76 sec.	0.62 sec.

Input2.txt

Threads

	1	2	4
Average Execution Time	7.88 sec.	7 sec.	5.97 sec.
Variance	0.72 sec.	0.53 sec.	0.015 sec.
Average Speedup	0	1.12 sec.	1.31 sec.

Input3.txt

Threads

	1	2	4
Average Execution Time	6.28 sec.	5.03 sec.	4.71 sec.
Variance	0.31 sec.	0.04 sec.	0.004 sec.
Average Speedup	0	1.24 sec.	1.33 sec.

Input4.txt

Threads

	1	2	4
Average Execution Time	86.95 sec.	72.79 sec.	57.95 sec.
Variance	37.64 sec.	33.61 sec.	2.14 sec.
Average Speedup	0	1.19 sec.	1.50 sec.

Input5.txt

Threads

	1	2	4
Average Execution Time	63.40sec.	54.44 sec.	41.19 sec.
Variance	36.8 sec.	13.52 sec.	1.46 sec.
Average Speedup	0	1.16 sec.	1.53 sec.

Υλοποίηση με Java Threads

Η λογική είναι ίδια με αυτή χρησιμοποιώντας tbb, αλλά τώρα αντί για parallel invoke έχουμε απλά παράλληλες ή σειριακές κλήσεις της computeMassCentre σε σχέση με τον αριθμό των νημάτων που έχουμε στην διάθεση μας και το βάθος του δένδρου για την αποφυγή των deadlocks. Επίσης αντί για parallel for τώρα χρησιμοποιείται μια custom thread pool όπου κάνει init τα worker threads και τρέχει tasks σε batches για καλύτερη διαχείριση των content switches. Όταν η επανάληψη των προσομοιώσεων τελειώσει, το thread pool γίνεται shutdown.

Πιο συγκεκριμένα, τα tasks **(όπου είναι η reset_force() , η applyForce(body,root) και η body.new_position() για κάθε πλανήτη)** τα παίρνει από μία BlockingQueue και γίνονται submit από την συνάρτηση simulation μετά από την κατασκευή του δένδρου και το υπολογισμό του κέντρου μάζας κάθε κόμβου.

Μετρήσεις για Java Threads:

Για κάθε input ισχύει:

5 runs.

Αριθμός επαναλήψεων προσομοίωσης = 10.000

Με χρήση της εντολής time παίρνοντας το real time.

Input1.txt

Threads

	1	2	4
Average Execution Time	1.74 sec.	2.30 sec.	3.58 sec.
Variance	0 .005 sec.	0.013 sec.	0 .021 sec.
Average Speedup	0	0.75 sec.	0.45 sec.

Input2.txt

Threads

	1	2	4
Average Execution Time	3.72 sec.	4 sec.	6.67 sec.
Variance	0.12 sec.	0.012 sec.	0.16 sec.
Average Speedup	0	0.92 sec.	0.55 sec.

Input3.txt

Threads

	1	2	4
Average Execution Time	3.51 sec.	3.83 sec.	6.06 sec.
Variance	0.02 sec.	0.003 sec.	0.015 sec.
Average Speedup	0	0.91 sec.	0.57 sec.

Input4.txt

Threads

	1	2	4
Average Execution Time	26.04 sec.	21.56 sec.	23.47sec.
Variance	1.83 sec.	0.21 sec.	0.60 sec.
Average Speedup	0	1.20 sec.	1.10 sec.

Input5.txt

Threads

	1	2	4
Average Execution Time	21.35 sec.	18.13 sec.	21.26 sec.
Variance	1.78 sec.	0.26 sec.	0.035 sec.
Average Speedup	0	1.17 sec.	1 sec.

Σύγκριση των δύο υλοποιήσεων σχετικά με:

Ευκολία υλοποίησης:

Στον σχεδιασμό και την ευκολία ανάπτυξης του αλγορίθμου μου άρεσε πιο πολύ η χρήση της βιβλιοθήκης TBB καθώς ο παραλληλισμός γίνεται πιο αυτοματοποιημένα με λίγες γραμμές χωρίς την ανάγκη για thread pools και manual graining των πλανητών σε batches για την καλύτερη διαχείριση των content switches.

Κλιμακωσιμότητα:

Η υλοποίηση με TBB πετυχαίνει μεγαλύτερη κλιμακωσιμότητα καθώς όσο αυξάνεται το input size και ο αριθμός των νημάτων οι χρόνοι εκτέλεσης μειώνονται, ενώ όταν το input size είναι πολύ μικρό όπως στο input1.txt, τότε όσο περισσότερα νήματα έχω τόσο πιο πολύ αυξάνεται ο χρόνος εκτέλεσης.

Από την άλλη στην υλοποίηση με Java Threads παρατηρούμε το ίδιο “μοτίβο” με παραπάνω αλλά όχι τόσο για πολλά νήματα (όπως 4 που παρουσιάζεται μια μικρή επιβράδυνση σε σχέση με τα 2 ή 3 που το έτρεξα εγώ). Παρ’ όλα αυτά τώρα βλέπουμε speedup καθώς αυξάνονται τα νήματα μόνο για πολύ μεγάλο αριθμό input data (like in input4.txt and input5.txt where I have 1000 and 1002 planets respectively, not 100 or 5 like in other input files).

Ταχύτητα εκτέλεσης:

Γενικά βλέπουμε ότι η υλοποίηση με TBB παρουσιάζει ταχύτερους χρόνους εκτέλεσης για όλα τα tests κάτι το οποίο είναι λογικό καθώς είναι μια βιβλιοθήκη σχεδιασμένη από ειδικούς και είναι όλα πολύ αυτοματοποιημένα σε σχέση με την υλοποίηση σε Java όπου υπάρχει custom thread pool και σχετικά απλό graining των tasks σε batches.

THE END

Ευχαριστώ για τον χρόνο σας!